



**Team
AXIOMATA**

AI DISCLOSURE AND DESIGN DECISIONS

Developed By
Team AXIOMATA

Developed for
Waypoint Group (Pvt) Ltd.

Access to RELAY

1. AI Use Disclosure

Axiomata used Claude (Anthropic) - in the Claude app and in Claude Code for VS Code - as a development accelerator throughout the build. This section describes exactly what AI produced, what the team did, and how every output was verified.

1.1 What AI Produced

Area	AI Contribution
Design prototypes.	HTML versions of the dispatcher desk and loader screens from the team's own design direction. Figma designs for the Designathon were made by the team.
Starter repository	First version of almost every source file - stack, roles, scope, design system, dispatcher concept - was written by Claude from the team's decisions, then run and tested end-to-end before handover.
Documentation	First drafts of the README and all files in docs/. This document is an example.

1.2 What the Team Did

- The product concept - 'one plan, four faces' - and all scope and priority decisions.
- Figma designs for all four roles and the complete design system.
- Every review, change and fix made after the AI-generated starter landed.
- QA on real phones and browsers, the field offline test, the fresh-clone test and the demo video.
- Sinhala and Tamil string review by native speakers.

1.3 How Output Was Checked

Verification Method	Detail
Rule validation	Every plan is checked by <code>validate_allocation()</code> - a port of the organisers' checker - in tests and CI. This found a real capacity rounding error that was then fixed with a regression test.
Behaviour tests	30 Python tests against a real Postgres; Playwright runs the full four-role walkthrough including a dark-van scenario.
Fresh-clone CI	docker compose up from a clean checkout; smoke tests must pass.
Data privacy	No competition data was sent to any external AI service. Only the six reference CSVs committed to git.

2. Design Departures from the Figma

Where the built product differs from the Designathon submission and the approved HTML prototypes, and why:

#	Screen	Design	Built	Reason
1	All	Separate apps per role	One app, one sign-in; account determines the face	One URL for judges; access enforced server-side
2	Dispatcher	Dark theme only	Dark and light themes (toggle in left rail)	Bright offices and projectors
3	Dispatcher	Static demo data	Live data from the planner	It is the working product, not a mock-up
4	Dispatcher	Clock in command bar	Virtual demo clock under the page title	Command bar overflowed at 1440 px with real counts
5	Loader	Tablet layout	Collapses to phone layout below 860 px	Loaders also use phones
6	Loader	—	Who's loading? crew picker on the shared tablet	Accountability on a shared device
7	Driver	Separate visual style	Rebuilt on loader's design system with light/dark themes	One family of field apps
8	Driver	English only	English, Sinhala and Tamil, switchable per account	Drivers' first languages
9	Driver	—	Offline banner, Sync tab and 'Test offline mode'	Honest states; safe demonstration of dark corridor
10	Store	Mobile frames	Desktop layout with sidebar ≥ 900 px; bottom tabs on phones	Store managers use both devices
11	Store	'Arrival time' before planning	'Set at 4 PM' until plan is published	No promise before a plan exists

3. Architectural Trade-offs

Decision	Reason	Cost Accepted
One FastAPI service serves API, SSE and SPA	Same-origin cookies; simple deploy; one URL	Web and API deploy together
Postgres LISTEN/NOTIFY for live updates	No extra infrastructure; works across Cloud Run instances	Notifications are signals, not a replayable log
SSE instead of WebSockets	One-way invalidation is sufficient; plain HTTP through Cloud Run	60-minute request limit; browsers reconnect
REST read models per face	Each screen loads one shaped payload; server is source of truth	Some refetching after each change
CP-SAT with time limit + explanation pass	Near-optimal plans in seconds with human-readable deferral reasons	Plans can differ slightly between runs
Outbox + idempotent sync instead of a sync framework	Small, testable, works with any backend	Conflict rules are hand-written (and documented)

Dispatcher desk as approved vanilla-JS prototype	Pixel fidelity to the Designathon-judged design	Two UI styles in one codebase
Virtual clock per workspace	Full delivery day demonstrated in minutes; sandboxes don't interfere	All business code must use <code>now_virtual()</code>

4. Scope Boundary

The following capabilities are outside Relay's scope for this competition:

- Stock levels and inventory management at depots or outlets - Relay plans and tracks orders, not stock.
- Route optimisation with real map data - the planning standard (`district_travel.csv`) is used as specified by the competition brief.
- Driver-to-dispatcher two-way chat - problem reports and notice feeds cover the necessary communication.

5. Next Steps (Post-Competition)

Areas the team identified for future development if Relay were taken to production:

- Real-time GPS tracking and automatic ETAs from device location rather than planning standards.
- Return-load and multi-day planning horizons.
- Automated stock-level integration for smarter order sizing.
- Push notifications (Web Push API) to supplement Server-Sent Events when the app is in the background.
- Expanded ML training pipeline as more delivery history accumulates.